OptiLang

A Visual Compiler and Optimization Explorer

Compiler Design Laboratory
Phase 3 Final Implementation and Testing

SHOURYA SARAF
24BDS0391

Faculty Ranjithkumar S
Academic Year 2026–27

Abstract

OptiLang is a small statically typed language and compiler that exposes its internal stages for teaching and review. The Phase 2 implementation already supported lexical analysis, recursive-descent parsing, an AST, semantic checks, a scoped symbol table, TAC, constant folding, and execution by an AST interpreter. Phase 3 adds assignments, while loops, typed functions and returns; TAC control-flow analysis; algebraic simplification and removal of selected unreachable branches; a stack bytecode compiler; a bounded virtual machine; and an offline visual explorer. On a successful run, the driver executes the original AST, optimized AST and bytecode, then compares their output and final global state. The included demonstration combines a loop and recursive factorial function and prints 120. An automated suite covers valid execution, errors, optimization, short-circuit behavior, limits and the generated explorer. The project provides an inspectable and testable compiler pipeline for its defined language subset. The equivalence checks cover executions that are run; they do not constitute a proof for every possible program.

1. Problem and objective

Compiler internals are difficult to follow when tokens, syntax trees, optimization decisions and runtime effects are shown by unrelated tools. OptiLang lets a student use one source program to inspect each representation and see why an optimization was applied. The final objective is a functioning source-to-bytecode compiler with static errors, predictable execution, and evidence that selected optimizations preserve observable results in the demonstrated programs.

2. Implemented language

The lexer recognizes keywords, identifiers, numeric and string literals, operators, punctuation and // comments with line and column positions. The parser accepts declarations, reassignment, expressions, print, blocks, if/else, while, typed top-level functions and calls. Recursive-descent precedence levels handle Boolean, equality, comparison, additive, multiplicative and unary operators. Types are int, float, bool and string.

The semantic analyzer rejects undefined or duplicate variables, type mismatches, non-Boolean conditions, incorrect function arguments, invalid returns and functions without an evident return path. Nested blocks have local scopes. A function may call another function before its definition, but functions themselves must be declared at top level.

3. Architecture and modules

Stage	Code unit	Inspectable result
Lexing and parsing	compiler.py	Tokens and AST
Semantic analysis	SemanticAnalyzer	Global symbols and controlled diagnostics
Intermediate representation	TACGenerator, cfg.py	Original/optimized TAC and basic-block successors
Optimization	ConstantFolder	Constant folding, algebraic identities, selected unreachable-code elimination and rule records
Target generation	bytecode.py	Stack instructions and bytecode CFG
Execution	VirtualMachine	Output and bounded instruction trace
Independent reference	Interpreter	Original and optimized AST output and final global state
Explorer	explorer.py	Standalone offline HTML with stage tabs

The bytecode compiler lowers the optimized AST directly. TAC is an inspectable side representation and is not the executable backend input. This boundary is explicit in the source and README.

4. Optimization and correctness checks

Constant folding evaluates literal-only operations. Algebraic simplification removes numeric + 0, - 0, * 1, and / 1. The optimizer eliminates a branch with a constant Boolean if condition, a while(false) body, and statements after a direct return. It records the rule and, for expression rewrites, the operator location. It does not apply unsafe identities such as x * 0 when evaluating x could itself fail or have effects. Boolean operators short circuit in the AST interpreter, TAC display, and bytecode VM.

The driver runs the unoptimized AST, optimized AST and VM. It raises a validation error if their outputs or final global variable states differ. This is a bounded, dynamic comparison on the input program, not formal translation validation. The VM limits total instructions to 100,000 and call depth to 100 by default. The reference interpreter also has execution limits.

5. Demonstration

Run from the repository root:

```
python -m pip install -e .
optilang examples/phase3.ol --html phase3-explorer.html
python -m unittest discover -s tests -v
```

The example increments count to five, calls recursive factorial(count), simplifies + 0, and prints 120. The console shows tokens, AST, symbols, original and optimized TAC, both CFG views, an optimization record, bytecode, VM trace, validation status and output. Opening the generated HTML displays these stages in one browser view.

6. Testing and evidence

The repository contains the original nine Phase 2 tests and 18 additional Phase 3 tests. The Phase 3 tests include recursion, forward calls, argument checking, loops, assignment, shadowing, global state, integer division and remainder, controlled division by zero, short-circuit evaluation, unreachable branches, instruction limits and HTML escaping. The command above was run in this implementation workspace and all 27 tests passed. The end-to-end examples/phase3.ol run exited successfully and printed 120. A recorded console run and a generated HTML explorer are available as output evidence.

7. Limitations

- TAC is displayed and analyzed, while bytecode is generated from the optimized AST rather than TAC.
- The visual explorer is a static, offline report. It does not support stepping through the VM interactively.
- The optimization record identifies the source operator and rewrite, but does not provide a persistent ID linking every source, TAC and bytecode instruction.
- SSA, phi insertion, dominator-based optimization, common-subexpression elimination, loop motion, register allocation, native code and formal equivalence proof are outside this implementation.
- Function return analysis is intentionally conservative. Some programs that would always return at runtime can be rejected.
- The VM trace is capped at 80 entries for readability. The instruction limit is separate and continues to apply.

8. Conclusion

OptiLang now compiles its defined language subset to bytecode and executes it in a custom VM while exposing the stages needed for a Compiler Design Laboratory demonstration. The Phase 3 implementation adds control flow, functions, more optimization, execution safeguards, a visual output and expanded tests. Its remaining advanced ideas are documented as limitations rather than represented as completed work.